

To install the binary, just drop it into a directory in your PATH, make Borg readable and executable for its users and then you can run it, as follows:

```
sudo cp borg-linux64 /usr/local/bin/borg
sudo chown root:root /usr/local/bin/borg
sudo chmod 755 /usr/local/bin/borg
```

Dependencies

To install Borg from a source package (including PIP), I had to install the following dependencies first:

- Python 3 >= 3.4.0, plus development headers. Even though Python 3 is not the default Python version on most systems, it is usually available as an optional install.
- OpenSSL >= 1.0.0, plus development headers.
- libacl and libattr (as libacl depends on libattr), plus development headers.

It has bundled code of the following packages, but Borg, by default (see *setup.py* if you want to change that) prefers a shared library if it can be found on the system (lib + dev headers) at build time:

- liblz4 >= 1.7.0 (r129)
- libzstd >= 1.3.0
- libb2

Some Python dependencies and Pip automatically installed themselves for me.

I followed the steps indicated at the Borg official site:

<https://borgbackup.readthedocs.io/en/stable/quickstart.html>.

A step-by-step example is given below:

```
1. #!/bin/bash
2. set -euo pipefail
3. IFS=$'\n\t'
4. #Written By Arvind GK
5. #Part 2 : To backup Second VM Disk
6.     This File name backup-vm-1st-disk.sh
7. DOMAIN="$1"
8. UUID=$(sudo virsh dumpxml $DOMAIN | grep uuid | cut
-d">" -f2 | cut -d"<" -f1)
9.
10. DISK_LOCATION=$(sudo virsh domblklist "$DOMAIN" |
grep hda | tr -s " " | cut -d" " -f2-)
11.
12. DISK_LOCATION1=$(sudo virsh domblklist "$DOMAIN" |
grep hdb | tr -s " " | cut -d" " -f2-)
13.
14. # Export Borg encryption key , You can also modify
this to using public and private key. Here I have user simple
passphrase as example.
15. export BORG_PASSPHRASE="1234"
16.
17. # Export the rate-limited remote shell
```

```
18. #export BORG_RSH="/usr/local/bin/pv-wrapper ssh"
19.
20.     Create the snapshot to a temp file on the Linux
drive
21.
22. sudo virsh snapshot-create-as --domain $DOMAIN \
23.
24. tempsnap "Temporary snapshot used while backing up
$DOMAIN" \ --disk-only --diskspec $DISK_LOCATION,file="/
img/$DOMAIN-tempsnap.qcow2" \ --atomic
25.
26.     Commit the changes that took place in the Windows
drive while
27.         Sync the virsh metadata with reality by
deleting the metadata
28.     external image right now but that is not
implemented yet)
29.
30. sudo virsh snapshot-delete "$DOMAIN" tempsnap
--metadata
31.
32.     (they have already been merged back into the
original image) sudo rm -f "/img/$DOMAIN-tempsnap.qcow2" sudo
rm -f "$DISK_LOCATION1"
33.
34. fi
35. }
36.
37.     Stupid hack: make a list of all the files we
*don't* want to save so we can exclude them from the backup
38.
39. find $(dirname "$DISK_LOCATION") ! -type d !
-wholenam "$DISK_LOCATION" | awk '{print "sh:" $0;}' > iso-
exclusions
40.     the --read-special flag tells borg to follow the
symlink & read the block device directly
41.
42.     Remove the temp file for the exclusions rm iso-
exclusions || /bin/true
43.     if it already succeeded and this ensures all
guest disk access is on
44.     Prune the backups on the server that are older
than a certain date echo "Pruning old backups"
45. borg prune -v --list /backup --prefix "$DOMAIN-"
--keep-within=1m & wait || /bin/true
46. echo "Done"
```

I modified this script by adding Borg to compress, encrypt and backup.

I used BLAKE2b for encrypting, as it had no legal implications on licensing and further, no history of anyone breaking the encryption.